Protocol-Mediated Agentic Domestic Money Movement Control Platform

Neil Khatu

1. SCOPE AND BUSINESS BOUNDARY **4**

2. EXPANDED DESIGN PRINCIPLES **5**

3. CONTROL PLANE DESIGN **6**

4. CAPABILITY SURFACE DESIGN **7**

4.1 Capability registry 7
4.2 Capability categories 8

5. CONTEXT AND MEMORY DESIGN **9**

5.1 State and Status Model 9
5.2 Task lifecycle 9
5.3 Delegation lifecycle 10
5.4 Error classes 10

6. PROTOCOL AND INTEROPERABILITY LAYER **11**

6.1 Layered model 11
6.2 Common message envelope 12
6.3 Standard message classes 12
6.4 Capability registry 12
6.5 Agent registry and trust tiers 12
6.6 Transport architecture 12
6.7 Trust, identity, and provenance 12
6.8 Versioning and compatibility 13
6.9 Reference implementation 13
6.10 Illustrative domestic payment release flow 13
6.11 Protocol layer definition for the target architecture 13

7 PROTOCOL INTEROPERABILITY REFERENCE MODEL **13**

7.1 Common message envelope 13
7.2 Standard message types 14
7.3 Agent card 15
7.4 Delegation contract 15
7.5 Agent trust tiers 15
7.6 Transport requirements 15
7.7 Recommended transport split 16
7.8 Delivery guarantees 16
7.9 Identity model 16
7.10 Delegated authorization token 16
7.11 Discovery and registry design 17

8. SECURITY POSTURE **18**

8.1 Provenance requirements 18
8.2 Policy checkpoints 19
8.3 Trust graph model 19
8.4 Operational guidance 19

8. REFERENCE WORKFLOW: DOMESTIC PAYMENT RELEASE **20**

9. OPERATING MODEL AND OBSERVABILITY **21**

10. NON-FUNCTIONAL REQUIREMENTS 22
11. REFERENCE TECHNOLOGY STACK 23
12. KEY CONFIGURATION MODEL 24
13. REPOSITORY AND CODE SETUP 25
14. EXAMPLE SERVICE CONTRACTS 26
15. IMPLEMENTATION ROADMAP 27
16. CONCLUSION 28
APPENDIX A. ARCHITECTURE READINESS CHECKLIST 29

Architecture Snapshot

a. Business boundary: Limited to domestic money movement such as ACH, same-day ACH, internal transfers, and RTP-style domestic rails.

b. Architectural thesis: Policy, action, and memory must be separated. If those boundaries collapse, ambiguity becomes financial risk.

c. Primary design objective: Enable autonomous execution without surrendering control over identity, approvals, side effects, and auditability.

d. • Non-negotiables: Task-scoped identity, deterministic controls outside the model, typed capabilities, explicit state, provenance, safe failure modes, and AI-native security engineering.

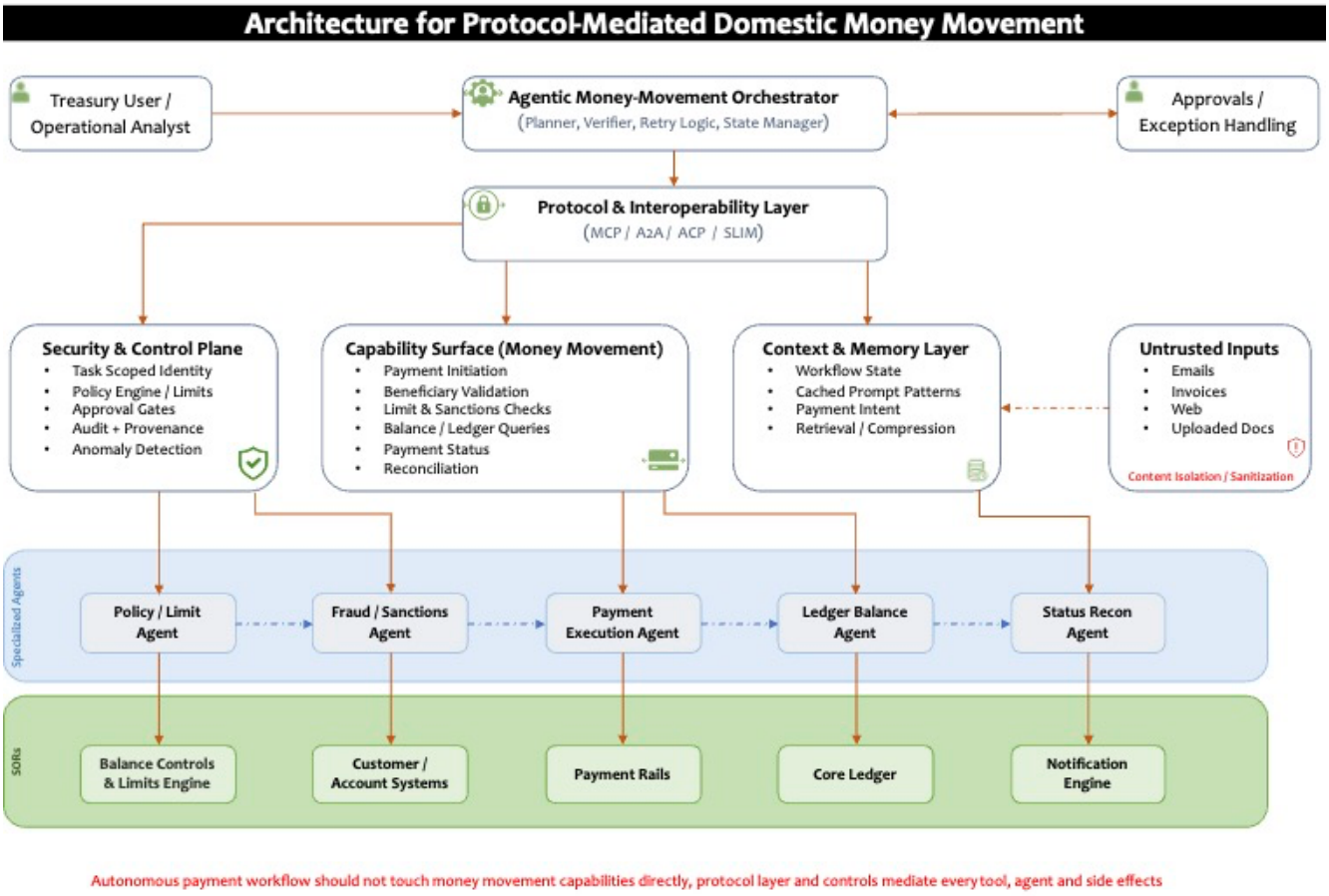


Figure 1. Reference architecture for an agentic domestic money movement platform.

1. Scope and business boundary

This architecture is intentionally bounded to domestic money movement. It covers customer-initiated and operations-assisted workflows such as payment drafting, validation, approval routing, release, exception handling, status inquiry, reconciliation support, and post-settlement investigation. It does not attempt to solve international wires, FX, card processing, sanctions policy design, or core ledger architecture in full.

That boundary matters. Domestic money movement is narrow enough to reason about concretely, yet high stakes enough that design mistakes are immediately visible. Vague status semantics can produce duplicate release attempts. Weak identity boundaries can over-authorize delegated agents. Poorly modeled memory can blur validated state with speculative reasoning. In this domain, architectural ambiguity is not just a design flaw. It is operational risk.

2. Expanded design principles

The platform should be designed around explicit architectural principles rather than implied engineering taste. These principles govern every layer: control plane, capability surface, context and memory, protocol mediation, and execution.

Principle	What it means in practice
Controls outside the model	The model may recommend an action, but it must not be the final policy engine for release authority, amount thresholds, approval requirements, destination restrictions, or data access.
Typed capabilities over vague endpoints	Agents handle narrow verbs and explicit schemas better than overloaded abstractions. Capabilities must describe required scopes, side effects, idempotency, retry behavior, and failure semantics.
State as a first-class asset	Workflow state should be durable, structured, versioned, and queryable. Payment status and approval state must never be inferred only from prior chat or prompt history.
Least privilege by task and time	Every user, agent, and delegated sub-agent should receive the smallest authority set required for the current task, with explicit expiration and bounded side effects.
Safe failure over speculative completion	When downstream systems respond ambiguously, the platform should prefer validated-hold, pending-reconcile, or escalate-for-review states over optimistic completion.
Idempotency at money-movement boundaries	All release-adjacent actions must be explicitly idempotent or guarded by deterministic duplicate prevention. This is essential for retries, partial failures, and delayed acknowledgements.
Provenance by default	Every consequential action should be reconstructable end to end: initiating user, parent agent, delegated agent, tool invoked, policy decision, and resulting side effect.
Trust-tiered memory	Trusted system instructions, enterprise records, workflow state, and untrusted external content must not be blended. Separate storage and retrieval policies are required.
Protocol-aware but policy-led	MCP, A2A, ACP, and SLIM improve interoperability and transport, but they cannot bypass local controls. Protocol participation must be subordinate to policy enforcement.
Human override for irreversible outcomes	Release, cancellation, approval override, beneficiary changes, and exception closure require deterministic gates and, where appropriate, human approval.
Observability as an operating requirement	Metrics, traces, approval latency, error classes, and replayable workflow events are part of the product. Without them, agentic execution is not production-ready.
Progressive containment	The platform must support read-only, dry-run, limited-rail, limited-beneficiary, and emergency downgrade modes before broad rollout.

3. Control plane design

The control plane is the governance system for agentic execution. It owns identity, authorization, policy enforcement, approval gating, risk controls, anomaly detection, rate limits, budget controls, kill switches, and the full audit trail for all consequential actions.

For domestic money movement, the control plane must answer operational questions before execution proceeds: who is asking, which agent is acting, what task is being attempted, which rail is in scope, whether the beneficiary is approved, whether release authority exists, whether the amount exceeds threshold, whether dual approval is required, and whether the action should proceed, pause, escalate, or be blocked.

The model must never be the final policy engine. A payment agent may recommend the next step, but deterministic policy services should make binding decisions on release authority, transaction limits, sensitive data access, and destination restrictions.

Domain	Representative controls	Expected output
Identity	user auth, agent auth, delegated credentials, scope TTL	authorized principal for the task
Policy	amount thresholds, rail rules, beneficiary status, approval matrix	allow / deny / escalate / simulate
Risk	anomaly scoring, unusual sequence detection, device or channel checks	risk posture and next action
Operations	rate limits, budget ceilings, kill switch, maintenance windows	safe execution envelope

4. Capability surface design

The capability surface is the machine-usable interface layer exposed to agents. It includes APIs, tool schemas, workflow contracts, protocol metadata, side-effect declarations, error taxonomies, and permissions metadata. Good endpoint design helps, but the real design target is the full capability surface through which an agent discovers and invokes work.

Capabilities should be narrow, typed, and explicit. Instead of a single vague action such as process_payment, the system should expose discrete operations such as create_domestic_payment_instruction, validate_beneficiary_account, request_payment_approval, release_approved_payment, check_payment_status, and cancel_pending_payment.

Every capability should declare preconditions, required scopes, irreversible side effects, idempotency behavior, retry expectations, timeout classes, approval dependencies, and structured failure semantics.

Capability	Type	Side effect	Required scope	Response contract
create_domestic_payment_instruction	draft	none	write:draft_payment	draft_id + normalized request
validate_beneficiary_account	validate	none	read:beneficiary validate:payment	validated / rejected / needs_review
request_payment_approval	approval	pending workflow entry	write:approval_request	approval_request_id + status
release_approved_payment	release	money movement	release:domestic_payment	release accepted / blocked / pending_reconcile
check_payment_status	status	none	read:payment_status	structured lifecycle state
cancel_pending_payment	cancel	possible workflow reversal	write:cancel_payment	cancel accepted / not cancellable

4.1 Capability registry

The capability registry is the machine-readable directory of everything an agent can call. Each entry must declare ownership, versioning, schemas, side effects, idempotency, required scopes, and retry behavior. This is the MCP-style contract layer.

```
id: domestic_payment.release_approved_payment
version: v1
description: Release an already approved domestic payment to execution
category: execution
side_effect_class: irreversible-financial-action
requires_human_approval: true
idempotency: required
allowed_callers:
  - agent.payment_orchestrator
  - agent.treasury_release_assistant
required_scopes:
  - payment.release
input_schema: schemas/release_payment_v1.json
output_schema: schemas/release_result_v1.json
timeout_ms: 5000
retry_policy: none
compensation_action: domestic_payment.raise_recall_request
```

```
observability:
  emit_events:
    - workflow.released
    - workflow.settlement_pending
```

4.2 Capability categories

Category	Representative capabilities
Inquiry	account balance, payment status, beneficiary info, cutoff lookup
Validation	beneficiary validation, policy validation, sanctions/fraud checks, rail eligibility
Approval	approval submission, approval status, escalation, exception routing
Execution	create payment instruction, release approved payment, cancel pending payment
Exception and reconciliation	raise exception, fetch exception state, reconcile payment status, request manual review

5. Context and memory design

The context and memory layer is the durable state substrate for multi-step execution. It should hold the task objective, workflow state, selected rail, validated beneficiary, approval status, payment identifiers, recent tool outputs, unresolved issues, and relevant business facts such as cut-off times or account restrictions. It is not a giant prompt dump; it is an explicit operating context.

Memory should be retrieval-based and scoped. Trusted system instructions, enterprise data, live workflow state, and untrusted external content should not be blended into a single undifferentiated window. They should be isolated by trust level, retention policy, and usage purpose.

A healthy architecture never forces the system to reconstruct payment state from chat history alone. If a same-day ACH payment for 85,000 USD has already passed beneficiary validation and is awaiting treasury approval, that state should be stored explicitly and versioned, not inferred from prior conversational turns.

Memory class	Examples	TTL / retention	Trust posture
Task state	rail, amount, beneficiary, current status	task lifetime + audit window	trusted workflow data
Reference data	cutoffs, account metadata, templates	hours to days	trusted enterprise data
Policy context	approval matrix, role rules, amount bands	versioned policy retention	trusted control data
External content	emails, tickets, uploaded docs	ephemeral / quarantined	untrusted until mediated

5.1 State and Status Model

A shared state model keeps orchestrators, sub-agents, operators, and downstream services aligned. Domestic payment workflows should not rely on chat history or ad hoc status strings.

Model	Statuses	Meaning
Task lifecycle	received, validated, awaiting_approval, approved, released, settlement_pending, settled, failed, cancelled, exception	Overall payment workflow state
Delegation lifecycle	offered, accepted, in_progress, waiting_on_dependency, completed, rejected, expired, failed	State of a delegated subtask between agents
Error classes	validation_error, policy_denied, approval_missing, transport_failure, timeout, duplicate_request, state_conflict, fraud_flagged, compliance_hold, capability_unavailable	Error taxonomy that lets agents recover intelligently

5.2 Task lifecycle

The task lifecycle defines the business state of a domestic payment workflow from intake through completion or failure. These states should be shared across agents, tools, and operator consoles.

1. received: request accepted but not yet validated.
2. validated: core request, beneficiary, and rail constraints passed initial validation.
3. awaiting_approval: payment is blocked pending one or more approval decisions.
4. approved: required approvals are present and still within validity window.
5. released: payment instruction has been sent to the execution path.
6. settlement_pending: release succeeded but final rail outcome is not yet confirmed.
7. settled: domestic payment completed successfully.
8. failed: execution, policy, or downstream processing failed terminally.
9. cancelled: task was intentionally stopped before irreversible release.
10. exception: workflow requires operator intervention due to ambiguity, conflict, or external error.

5.3 Delegation lifecycle

The delegation lifecycle defines how a parent agent offers work to a sub-agent and how bounded delegated work progresses over time.

1. offered: parent agent sends a bounded delegation request.
2. accepted: sub-agent accepts scope, deadline, and expected output contract.
3. in_progress: delegated work is actively executing.
4. waiting_on_dependency: delegated work is paused on an approval, policy decision, or downstream dependency.
5. completed: sub-agent returned a valid result within scope.
6. rejected: sub-agent refuses the task because scope, trust tier, or capabilities do not match.
7. expired: delegation token or task TTL expired before completion.
8. failed: delegated execution could not complete because of policy denial, timeout, or downstream error.

5.4 Error classes

Agents should not receive a generic invalid request. The protocol layer needs an explicit error taxonomy so orchestrators can recover safely and operators can diagnose quickly.

1. validation_error: malformed or incomplete request data.
2. policy_denied: the control plane rejected the action under current policy.
3. approval_missing: required approval is absent, stale, or insufficient.
4. transport_failure: message delivery failed or callback channel is unavailable.
5. timeout: agent, capability, or downstream service exceeded allowed latency budget.
6. duplicate_request: idempotency violation or replay detected.
7. state_conflict: requested action is incompatible with current task state.
8. fraud_flagged: fraud controls require hold or escalation.
9. compliance_hold: sanctions or compliance process blocked progress.
10. capability_unavailable: required capability is unhealthy, deprecated, or unreachable.

6. Protocol and interoperability layer

Protocols matter because the system will not remain a single-agent monolith for long. Model Context Protocol (MCP) enables standardized exposure of tools and context. Agent-to-Agent (A2A) patterns enable delegation to specialized remote agents. Agent Communication Protocol (ACP) contributes interoperability concepts for agent coordination. Secure Low-Latency Interactive Messaging (SLIM) represents the transport-level reality that serious agent systems need more than raw HTTP request-response semantics.

The design implication is that APIs are no longer the only interface surface that matters. The platform must be usable through protocol-mediated capability ecosystems. Tool descriptions, trust boundaries, caller constraints, state contracts, and side-effect declarations all need to be legible to machines and enforceable by the surrounding control plane.

Purpose. This layer standardizes how agents, tools, workflows, and downstream banking services discover one another, exchange work, preserve delegated authority, and move messages safely across the domestic money movement platform. It is intentionally separate from the control plane and the capability surface: it does not decide policy and it does not own payment logic. It makes multi-agent execution interoperable, bounded, observable, and replaceable.

6.1 Layered model

Layer	Primary responsibility	Preferred protocol pattern	Domestic money movement examples
Capability interface	Expose callable tools, workflows, and context contracts	MCP-style capability contracts	Validate beneficiary, get ACH cutoff, submit payment for approval, release approved payment
Agent delegation	Pass bounded work to specialized agents	A2A / ACP-style delegation	Compliance screening, treasury policy validation, exception handling, approval orchestration
Transport and delivery	Move requests, events, callbacks, and state changes	SLIM-like transport abstraction over HTTP/gRPC/NATS/Kafka	Approval events, settlement updates, release callbacks, reconciliation fan-out
Trust and provenance	Bind identity, scopes, traceability, and policy hooks	Signed envelopes + delegated tokens + audit metadata	Human initiator, parent agent, delegated agent, and executing service all remain visible

Design principles

1. **Explicitness:** Every message must carry typed schemas, declared side effects, retry semantics, and approval expectations.
2. **Delegation with boundaries:** Sub-agents receive task-bounded, time-bounded, and action-bounded authority instead of inheriting broad user power.

3. **Transport independence:** Semantic contracts survive transport changes; the business meaning of a release request remains stable whether the payload rides over SLIM, NATS, Kafka, or HTTPS.
4. **Async-first execution:** Domestic money movement includes approvals, cutoffs, settlement events, and exceptions, so asynchronous workflows are treated as normal.
5. **Security outside the model:** The protocol layer carries policy hooks and identity context, but deterministic policy engines remain the final authority for irreversible actions.

6.2 Common message envelope

All protocol interactions should share a canonical envelope that includes message_id, correlation_id, task_id, workflow_id, message_type, timestamp, source principal, delegation chain, target, trust metadata, payload, and distributed trace identifiers. This keeps agent frameworks interchangeable while preserving end-to-end lineage.

6.3 Standard message classes

The platform should distinguish capability.discover, capability.invoke, capability.result, agent.delegate, agent.accept, agent.progress, agent.complete, policy.decision.request, approval.requested, and workflow state events such as awaiting_approval, released, settlement_pending, and exception_raised. Narrow verbs help agents recover cleanly and help operators reason about failure.

6.4 Capability registry

Each callable capability publishes a machine-readable descriptor with owner, version, input and output schema, side-effect class, idempotency requirement, required scopes, timeout, retry policy, and compensation action. In domestic money movement, release_approved_payment should be declared as an irreversible financial action with human approval and idempotency required.

6.5 Agent registry and trust tiers

Each internal or remote agent publishes an agent card describing its purpose, supported tasks, accepted scopes, trust tier, transport support, callback mode, and maximum task lifetime. Most payment sub-agents should remain Tier 1 read-only or Tier 2 validation-only; execution authority should be rare and always policy-gated.

6.6 Transport architecture

Use synchronous request-response for low-latency lookups such as balance inquiry, beneficiary validation, and policy decisions. Use asynchronous delivery for approval workflows, release pipelines, settlement updates, and reconciliation events. Delivery guarantees should be explicit: at-least-once with idempotent business consumers for most workflow messages, and exactly-once business effects enforced through idempotency keys and state transition guards rather than transport mythology.

6.7 Trust, identity, and provenance

Every envelope should preserve four identities: human initiator, parent agent, delegated sub-agent, and executing capability or service. Delegated authorization tokens should carry task ID, scopes, allowed rails, amount ceilings, approval flags, beneficiary constraints, and expiry. Every irreversible step must be reconstructable from audit trails without relying on model memory or chat history.

6.8 Versioning and compatibility

The interoperability layer should support explicit version negotiation for capabilities, agent cards, and message envelopes. Deprecated contracts must advertise retirement dates and supported successors so orchestrators do not discover breaking changes at run time.

6.9 Reference implementation

A practical implementation uses JSON Schema for contracts, OpenAPI for HTTP surfaces, signed JWT or SPIFFE-based delegated identities, OpenTelemetry traces, OPA policy hooks, a registry service for agents and capabilities, and a transport adapter that can target SLIM, NATS, Kafka, gRPC, or HTTPS while keeping message semantics stable.

6.10 Illustrative domestic payment release flow

1. Treasury user submits a domestic ACH or RTP payment request.
2. Orchestrator loads task context and queries the capability registry for beneficiary validation, cutoff lookup, and payment drafting contracts.
3. A compliance agent receives a bounded A2A-style delegation request to perform screening with no execution authority.
4. The control plane evaluates amount threshold, beneficiary status, rail eligibility, and approval policy.
5. An approval agent raises an approval.requested event and waits asynchronously for a release decision.
6. Once approved, the orchestrator invokes release_approved_payment through the capability surface using a task-scoped delegated token.
7. Transport emits release and settlement events, and the full provenance chain is written to the audit stream.

6.11 Protocol layer definition for the target architecture

The protocol and interoperability layer is the machine-to-machine coordination plane that standardizes capability discovery, agent delegation, message exchange, trust propagation, and compatibility across the domestic money movement platform. It combines MCP-style capability contracts, A2A and ACP-style delegation semantics, SLIM-like transport abstractions, task-scoped identity, and policy-aware provenance so that agents, tools, and banking services can interoperate safely under financial control constraints.

7 Protocol Interoperability Reference Model

This section turns the protocol and interoperability layer into an implementation-grade reference. It adds the canonical message envelope, standard message classes, capability registry contract, agent card, delegation contract, trust tiers, transport requirements, delivery guarantees, identity model, delegated authorization token, and the detailed state/status model needed for domestic money movement.

7.1 Common message envelope

Every protocol interaction should use one canonical envelope so that agents, tools, and downstream services can preserve lineage, policy context, and compatibility regardless of transport.

```
{
  "message_id": "msg_01J2Q7A8C4B6D9",
  "correlation_id": "corr_dompay_8472",
  "task_id": "task_domestic_payment_122",
  "workflow_id": "wf_ach_release_443",
  "message_type": "capability.invoke",
  "timestamp": "2026-03-20T15:04:05Z",
```

```
  "source": {
    "principal_type": "agent",
    "principal_id": "agent.payment_orchestrator",
    "tenant_id": "bank_us_treasury",
    "session_id": "sess_72A9"
  },
  "delegation": {
    "delegated_by": "agent.treasury_assistant",
    "delegation_chain": [
      "user.neil",
      "agent.treasury_assistant",
      "agent.payment_orchestrator"
    ],
    "scope": [
      "payment.validate",
      "payment.submit_for_approval"
    ],
    "expires_at": "2026-03-20T15:19:05Z"
  },
  "target": {
    "target_type": "capability",
    "target_id": "domestic_payment.validate_beneficiary",
    "version": "v1"
  },
  "trust": {
    "classification": "internal-sensitive",
    "human_approval_required": false,
    "policy_context_id": "polctx_337"
  },
  "payload": {},
  "trace": {
    "trace_id": "tr_9b21",
    "span_id": "sp_17aa"
  }
}
```

7.2 Standard message types

Class	Message types	Use in domestic money movement
Capability	capability.discover, capability.describe, capability.invoke, capability.result, capability.error	Discover tools, invoke beneficiary validation, get ACH cutoffs, return status or errors
Agent delegation	agent.discover, agent.describe, agent.delegate, agent.accept, agent.reject, agent.progress, agent.complete, agent.fail	Hand bounded work to compliance, risk, approvals, or exception agents
Workflow and events	workflow.started, workflow.awaiting_approval, workflow.approved, workflow.released, workflow.settlement_pending, workflow.settled, workflow.exception_raised	Track long-running domestic payment state transitions
Security and policy	policy.decision.request, policy.decision.result, approval.requested, approval.granted, approval.denied, risk.flagged	Capture gating decisions before irreversible actions

7.3 Agent card

Every internal or remote agent publishes an agent card that makes delegation safe and predictable. This is the A2A/ACP-style discoverability contract.

```
agent_id: agent.compliance_screening
name: Compliance Screening Agent
purpose: Screen domestic payments and beneficiaries for policy and compliance holds
supported_tasks:
  - payment.compliance_check
  - beneficiary.screening
accepts_scopes:
  - payment.validate
trust_tier: tier-2-controlled
supported_transports:
  - slim
  - https
callback_mode: async
max_task_ttl_seconds: 900
requires_provenance: true
returns_schema: schemas/compliance_result_v1.json
execution_authority: none
```

7.4 Delegation contract

Delegation must always be bounded. A parent agent should never issue vague instructions such as handle this payment. The contract should define intent, scope, output schema, deadline, policy requirements, and the escalation path.

- **Required fields:** task intent, granted scope, input context, expected output schema, deadline, approval requirements, escalation path, provenance requirements.
- **Good example:** Validate beneficiary and return validation_result_v1 within 30 seconds with no execution authority.
- **Bad example:** Handle this ACH payment end to end.

7.5 Agent trust tiers

Tier	Authority	Typical domestic money movement use
Tier 1	Read-only; no side effects	balance lookup, cutoff lookup, payment status inquiry
Tier 2	Validation and recommendation only	compliance screening, fraud scoring, beneficiary validation
Tier 3	Can initiate approval workflows only	approval routing, exception escalation
Tier 4	Can trigger execution only with policy and approval gates	payment release orchestrator
Tier 5	Restricted system-level agent	rare operational break-glass or supervised recovery flows

7.6 Transport requirements

- **Core requirements:** low-latency request/response, asynchronous callback, reliable delivery, correlation IDs, authentication, message signing, encryption, backpressure, retry policy, dead-letter handling.

- **Why it matters:** domestic money movement includes approvals, release callbacks, settlement updates, and reconciliation exceptions, so the transport layer must support both real-time lookups and long-lived event flows.

7.7 Recommended transport split

Mode	Use cases	Recommended transport pattern
Synchronous	balance inquiry, beneficiary validation, policy decision checks	request/response over HTTPS, gRPC, or SLIM request mode
Asynchronous	approval workflows, release pipeline, settlement updates, exception handling, reconciliation	event-oriented messaging over SLIM, NATS, Kafka, or equivalent bus

7.8 Delivery guarantees

- **At-most-once:** only for harmless advisory notifications where loss is acceptable.
- **At-least-once:** default for most workflow events, paired with idempotent consumers and business deduplication.
- **Exactly-once business effect:** achieved through idempotency keys, immutable event IDs, and state transition guards, not by trusting transport claims alone.

7.9 Identity model

The protocol layer must preserve four identities on every material action so that trust and accountability survive delegation.

Identity	Meaning
Human initiator	The person or system that requested the domestic payment action
Parent agent	The agent that owns the current workflow and makes the delegation decision
Delegated sub-agent	The agent receiving bounded authority for a specific task
Capability or service executor	The concrete service or tool that performs the action

7.10 Delegated authorization token

Delegated tokens carry the bounded authority for a single task or subtask. They should be time-limited, action-limited, and explicit about financial constraints.

```
{
  "sub": "agent.payment_orchestrator",
  "act": "on_behalf_of:user.neil",
  "task_id": "task_122",
  "scopes": ["payment.validate", "payment.submit_for_approval"],
  "max_amount_usd": 100000,
  "allowed_rails": ["ach", "rtp"],
  "beneficiary_constraints": ["approved-only"],
  "requires_dual_approval": true,
  "exp": 1774022400
}
```

7.11 Discovery and registry design

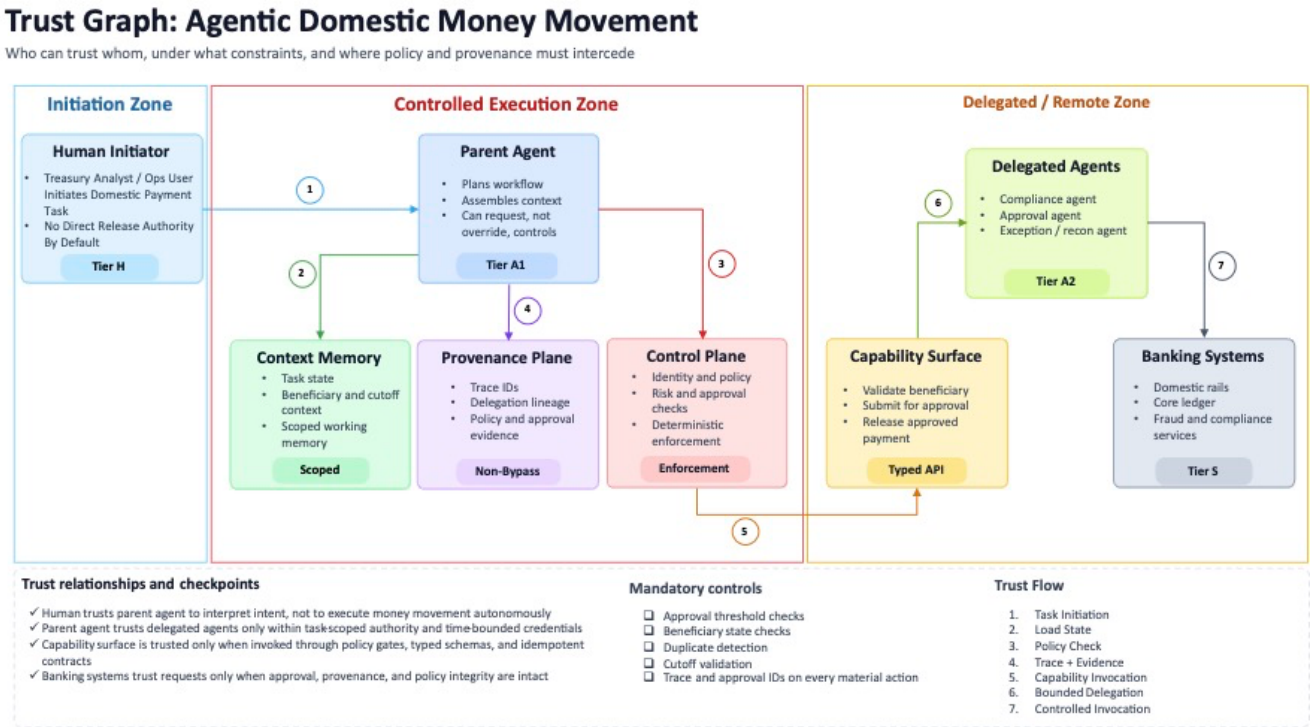
The platform should maintain separate but linked registries for capabilities and agents. Discovery must be machine-readable, versioned, and policy-aware; agents should not scrape prose documentation to determine what can be called.

1. Capability registry responsibilities: publish tool and workflow descriptors, schema links, side-effect class, retry policy, required scopes, owner, health, and deprecation state.
2. Agent registry responsibilities: publish agent cards, trust tiers, supported task types, accepted scopes, callback endpoints, supported transports, and tenancy boundaries.
3. Lookup behavior: allow search by task type, rail, side-effect class, trust tier, and environment so orchestrators can choose the correct target automatically.
4. Health and compatibility: registries should expose version compatibility, current health, rollout stage, and replacement targets for deprecated capabilities or agents.
5. Policy linkage: registry entries should reference the policy package, approval profile, and data classification that apply to the capability or agent.
6. Governance: only signed and approved registry entries may be discoverable in production; shadow or experimental entries should be isolated by environment.

8. Security posture

Security in an agentic money movement platform is not an API wrapper concern. It is a control architecture concern. The threat model includes prompt injection, over-broad delegated authority, unsafe retries, unauthorized side effects, state confusion, and loss of provenance across multi-agent execution.

1. Task-scoped credentials for user sessions, orchestrators, and delegated sub-agents.
2. Read-only, simulation, and release-gated modes for sensitive capabilities.
3. Anomaly detection for unusual tool sequences, payment patterns, and escalation loops.
4. Immutable lineage from initiating user through orchestrator, sub-agent, tool invocation, policy decision, and resulting side effect.
5. Policy-aware retry rules so ambiguous downstream responses do not cause duplicate money movement.
6. AI security engineering built into the baseline stack: prompt isolation, context compartmentalization, tool authorization middleware, side-effect sandboxes, and adversarial evaluation.



8.1 Provenance requirements

The interoperability layer must preserve enough provenance to reconstruct what happened without relying on chat history or implicit model reasoning. Provenance should be written to both the live trace and the immutable audit stream.

1. Initiation lineage: initiating user, channel, session, and original business request.
2. Delegation lineage: parent agent, delegated agent, scope granted, token identifier, and expiry.
3. Context lineage: which enterprise records, cutoffs, policies, and retrieved artifacts informed the decision.
4. Policy lineage: policy package version, decision result, overrides, and approval evidence.
5. Execution lineage: exact capability invoked, version, target system, idempotency key, and downstream response.

6. Outcome lineage: final state transition, settlement or failure outcome, and operator-visible explanation for the action.

8.2 Policy checkpoints

Every irreversible or financially material action must pass deterministic policy checkpoints before execution. These checks should happen outside the model and before any release, cancel, beneficiary change, approval escalation, or rail switch.

1. Authority checks: initiator role, delegated agent scope, dual-control requirement, and release entitlements.
2. Payment checks: amount thresholds, rail eligibility, same-day ACH and RTP rules, cutoff windows, and domestic-only restrictions.
3. Beneficiary checks: approved counterparty status, account validation outcome, recent maintenance changes, and duplicate beneficiary risk.
4. Risk and compliance checks: fraud signals, sanctions/compliance screening result, behavioral anomaly score, and unresolved exception flags.
5. State checks: current workflow state, approval status, stale-context detection, idempotency key reuse, and conflicting in-flight actions.
6. Execution checks: allowed side-effect class, compensation path availability, and explicit confirmation that the action is reversible or irreversible.

8.3 Trust graph model

In domestic money movement, trust is not a single yes-or-no property. It is a graph of principals, artifacts, tools, and side effects. The protocol layer should model and enforce that graph explicitly.

1. Nodes: human users, parent agents, sub-agents, capabilities, policy engines, context sources, approval actors, and downstream banking services.
2. Edges: delegation, invocation, approval, data retrieval, event emission, and side-effect execution.
3. Trust attributes: trust tier, data sensitivity, allowed scopes, approval requirement, and transport assurance level.
4. Graph rules: no edge may silently widen authority; every delegation edge must narrow or preserve scope, never expand it.
5. Runtime use: the control plane should evaluate the active trust graph before allowing release, cancellation, or exception override actions.

8.4 Operational guidance

1. **Preferred sequencing:** user request -> context load -> capability discovery -> validation calls -> delegation -> policy decision -> approval -> release -> settlement events -> audit closeout.
2. **Failure posture:** when provenance is missing, approval state is ambiguous, or policy context is stale, degrade safely to pause, deny, or manual review.
3. **Implementation note:** MCP shapes capability contracts, A2A and ACP shape delegation semantics, SLIM or transport adapters carry messages, and the control plane remains the final authority for irreversible financial actions.

8. Reference workflow: domestic payment release

A representative workflow begins when a user or upstream treasury system requests a domestic payment. The context layer assembles the active state, relevant account and beneficiary data, applicable payment rail, and operational constraints. The orchestrator decides the next step. The control plane evaluates identity, authority, amount thresholds, beneficiary status, and approval requirements. Only then does the capability surface expose the exact validation or release action permitted for that task.

If the payment passes validation but still requires approval, the workflow should update state to validated-awaiting-approval and stop. If approval arrives, the release capability becomes available. If a downstream system returns a partial or ambiguous response, the system should prefer a safe holding state over speculative completion. The control plane should then determine whether to retry, pause, reconcile, or escalate.

9. Operating model and observability

A production operating model should assume that agentic execution is an ongoing operational system, not a feature. That requires end-to-end traces, decision logs, policy logs, cost telemetry, queue metrics, capability health, memory freshness indicators, approval latency, and per-rail execution dashboards.

Platform engineering should own capability contracts and runtime health. Security should own policy logic, trust boundaries, and evaluation. Payments operations should own exception semantics and business controls. AI engineering should own orchestration behavior, model usage, prompt hygiene, and evaluation of recovery behavior under failure.

10. Non-functional requirements

1. Predictable latency classes and bounded queues for interactive vs background work.
2. Strong idempotency for release-adjacent actions and deterministic retry contracts.
3. Immutable event history and replayable workflows for investigations and controls testing.
4. Versioned capability contracts with backwards-compatible rollouts.
5. Clear separation between synchronous user-facing operations and asynchronous background processing.
6. Progressive containment modes: read-only, dry-run, single-rail isolation, restricted beneficiary lists, emergency downgrade.

11. Reference technology stack

The stack below is intentionally practical. It favors mainstream components that can be built, secured, and operated today in a bank or fintech environment, while leaving room for protocol-mediated agent patterns.

Layer	Recommended stack	Why it fits	Config priorities
UI + ops console	React / Next.js	banking ops workflows, review queues, approvals	SSO, read-only mode, field-level masking
API gateway	Kong or Apigee	central auth, quota, policy hooks	mTLS, JWT auth, rate limits, request IDs
Orchestrator	Python FastAPI or TypeScript NestJS	strong ecosystem for agent orchestration	strict tool registry, retry policy, timeout budgets
Policy engine	Open Policy Agent (OPA)	deterministic policy outside the model	bundle versioning, signed policies, explain logs
Workflow engine	Temporal	durable multi-step execution and replay	workflow IDs, task queues, retry classes, compensation rules
Message / event bus	Kafka or NATS; SLIM where appropriate	asynchronous state propagation and agent transport patterns	topic ACLs, retention, schema registry
Context store	PostgreSQL + Redis + vector store	explicit state + cache + retrieval	TTL, namespace isolation, encryption, provenance tags
Observability	OpenTelemetry + Prometheus + Grafana	traces, metrics, dashboards	trace propagation, cost tags, rail tags
Secrets / identity	Vault + bank IAM / Okta / Entra	scoped credentials and key rotation	short-lived creds, service identities, audit logs
Deployment	Kubernetes	policy-driven deployment and isolation	network policy, pod identity, runtime limits

12. Key configuration model

A successful build needs opinionated defaults. The following configuration domains should exist from day one so security and operating controls are not buried in prompt text or ad hoc code paths.

```
environment:
  name: prod-us-east
  region: us-east-1
  rail_scope: [ach, same_day_ach, internal_transfer, rtp]
  default_mode: read_only
control_plane:
  policy_engine: opa
  dual_approval_threshold_usd: 50000
  high_risk_escalation_threshold_usd: 100000
  max_release_retries: 0
  ambiguous_response_action: pending_reconcile
  kill_switch_enabled: true
capabilities:
  release_approved_payment:
    requires_scope: release:domestic_payment
    approval_required: true
    idempotency_key_required: true
    dry_run_supported: false
  validate_beneficiary_account:
    requires_scope: validate:payment
    approval_required: false
memory:
  task_ttl_minutes: 240
  external_content_quarantine: true
  provenance_required: true
observability:
  trace_sampling: 1.0
  payment_event_topic: payments.events
  policy_decision_topic: policy.decisions
security:
  token_ttl_minutes: 15
  require_mtls_internal: true
  pii_redaction_logs: true
```

13. Repository and code setup

The implementation should be split by control domain so policy, workflow state, and tool execution do not collapse into a single service. A simple starting repository layout is below.

```
repo/
  apps/
    ops-console/            # approvals, review queues, investigations
    orchestrator-api/       # task intake, tool routing, agent execution
    policy-service/         # OPA bundles, decision API, explain logs
    capability-gateway/     # typed capability wrappers around payment systems
  services/
    workflow-worker/        # Temporal workers for validation/release flows
    event-consumer/         # Kafka/NATS consumers for state updates
    memory-service/         # task state, retrieval, provenance tags
  packages/
    capability-schemas/     # shared JSON Schema / OpenAPI / protobuf defs
    policy-models/          # role maps, threshold configs, rule tests
    security-middleware/    # authz, redaction, audit, idempotency helpers
    observability/          # OpenTelemetry setup, metric tags, log schema
  infra/
    kubernetes/
    terraform/
    helm/
  docs/
    architecture/
    runbooks/
    threat-models/
```

Starter services should include unit tests for policy decisions, contract tests for each capability, replay tests for workflow execution, and adversarial tests for ambiguous or malicious tool inputs. Shipping an agentic payment platform without these test classes is equivalent to shipping payment code without reconciliation.

14. Example service contracts

```
# FastAPI - capability wrapper
@router.post("/payments/{payment_id}/release")
async def release_payment(payment_id: str, req: ReleaseRequest, principal: Principal):
    enforce_scope(principal, "release:domestic_payment")
    require_idempotency_key(req.idempotency_key)
    policy = policy_client.evaluate(
        action="release_approved_payment",
        payment_id=payment_id,
        principal=principal,
    )
    if not policy.allow:
        raise HTTPException(status_code=403, detail=policy.reason)
    return await payment_service.release(payment_id, req)
```

```
# Temporal workflow - safe state transition
with workflow.step("release-approved-payment"):
    status = payments.get_status(payment_id)
    if status != "approved":
        return hold_state("awaiting_approval")
    result = capabilities.release_approved_payment(payment_id, idempotency_key)
    if result.state in {"ambiguous", "partial"}:
        return hold_state("pending_reconcile")
    return persist_state("released")
```

15. Implementation roadmap

- Phase 1 should establish typed capabilities, explicit workflow state, and a deterministic control plane around a narrow domestic payment use case.
- Phase 2 should introduce protocol-mediated tool access, richer observability, and structured approval workflows.
- Phase 3 should add specialized agents, remote delegation patterns, and stronger runtime security controls such as anomaly scoring and side-effect sandboxes.

The critical mistake is trying to start with a general-purpose agent. The safer path is to begin with constrained, high-observability workflows where every state transition, authority boundary, and tool call can be reasoned about and audited.

16. Conclusion

The architecture for agentic domestic money movement cannot be a lightly modified developer stack. It requires explicit separation between governance, capabilities, state, and execution. It requires protocol awareness without surrendering control to protocols. And it requires a security posture that treats autonomous behavior as a first-class systems problem.

The practical question is no longer whether agents can call a payment API. It is whether the platform can remain controllable, auditable, and safe when machines become the operational consumers of domestic money movement. That is the architecture bar this document is designed to meet.

Appendix A. Architecture readiness checklist

1. Every consequential capability has explicit input/output schemas and side-effect semantics.
2. No irreversible payment action can execute without deterministic policy review outside the model.
3. Workflow state is persisted as structured data rather than inferred from conversation history.
4. Agent and delegated-agent credentials are scoped to task, time, and allowed action set.
5. All payment workflows produce end-to-end provenance that can be reconstructed after the fact.
6. Read-only, dry-run, and restricted-operating modes exist before broad production rollout.
7. Protocol-mediated access through MCP or A2A does not bypass policy, logging, or approval gate